FRED TALKS

13th November 2026

CONTENTS

What requirements don't you have?

SEANGOEDECKE.COM RSS FEED · 945 WORDS

Please for the love of all that is holy: Stop writing long boring Titles

SWYX · 1431 WORDS

Ownership

THORSTEN BALL · 1122 WORDS

The lies I used to tell myself

CATE HALL · 1900 WORDS

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 1053 WORDS

What requirements don't you have?

SEANGOEDECKE.COM RSS FEED · 19 OCT 2026 · SOURCE

When designing software, the requirements you don’t have are as important as the requirements you do. Identifying what you can compromise on — what can be slow, or stale, or inflexible — is usually the key to an elegant design.

Elegant systems drop some requirements

[SVG OMITTED]

For instance, take preforking web servers like Unicorn. Because they lack the feature of shared mutable state between server instances, they can run each server in its own process, which provides isolation for free. Because Unicorn specifically ignores the problem of slow client handling, it can queue requests directly on Unix sockets, drastically simplifying the implementation.

Because S3 does not support atomic multi-key updates — you cannot update multiple blobs in a single transaction — it can store data at the blob level however it chooses. That makes it much faster, since reads and writes don’t have to cross storage partitions, and much easier to scale, since less coordination means fewer sources of truth.

StatsD uses UDP by default, which means it doesn’t guarantee that every piece of data you send makes it into storage. In exchange for dropping that requirement, it incurs much less network overhead, which is a big advantage for a stats aggregation service that’s constantly shoveling data over the wire.

The general design approach of fanning out data to an edge cache buys a lot of speed and scaling by dropping the requirement that data must always be up-to-date. SQLite buys operational simplicity — it can run out of a single file! — by dropping the requirement of concurrent writes. By accepting high tail latency, Cursor can scale their distributed git backend more easily. Well-designed systems turn their weaknesses into strengths.

Why dropping requirements is hard

[SVG OMITTED]

Anyone who's spent time designing real-world software (or who's studied distributed systems) knows that system design is a process of compromise. If you attempt to satisfy every single potential requirement — speed, concurrency, up-to-date data, a simple interface, supporting complex user behaviors — you will fail to design a working system at all.

Figuring out what requirements you can relax is difficult because **it combines both business and technical expertise**. It is technically challenging to identify precisely the level of staleness that a particular design change will allow; it is likewise challenging from a business perspective to determine whether that level of staleness is acceptable to users.

In most cases, an engineer who understands the technical side must try and explain it to a product manager, who can make a judgement on the business side. But this is a very difficult partnership to get right. Engineers and PMs have different incentives and little ability to check each other's work, which makes it hard to build trust. Without trust, the product manager is liable to avoid giving any ground at all, which leaves the design in an impossible state.

This is an underappreciated reason why solo developers or startups can sometimes run rings around larger companies. Because the technical and business decisions are united in a single person, they can make design tradeoffs that would be impossible for a large company, enabling them to build systems that are far more elegant and powerful.

Working with LLMs

[SVG OMITTED]

Knowing what requirements you don't have becomes far more important when working with coding agents. Frontier models are very capable, but they have all gone through the purifying flames of RLVR: like high-achieving schoolchildren, they are obsessed with satisfying the Grader. On top of that, they have very little business context compared to an experienced human. That often manifests as them catching correctness or consistency "bugs" that are really just hypothetical unsatisfied requirements.

For instance, it's not uncommon for an LLM to casually introduce a mutex or other concurrency mechanism to ensure that two pieces of semi-related data are kept in sync, even if not told to do so. I've seen LLMs add weird checks in the middle of request handling to ensure that some already-checked piece of data hasn't changed mid-flight, even if that wouldn't successfully prevent the feared TOCTOU bug. LLMs are obsessed with edge cases like "what if the current

user loses their admin status in the middle of us handling one of their admin requests?", or "what if the cache in storage partition A is updated a nanosecond before the one in storage partition B, leading some users to see different data?". Particularly if run in a loop or told to "make no mistakes", they will pile on cases like these until they've transformed your four-line change into a thousand-line monster.

This is why I'm confused by some people's claims that code review is dead. About half the time I review a large AI-generated change, I notice some unnecessary requirement that makes the diff more complex. When I dig deeper, the LLM invariably introduced this requirement on its own.

When working with a cache, I will often spell out in my prompt that I don't mind the data sometimes being stale or briefly inconsistent between cache instances. When reviewing Golang code, the first thing I do is scan for added mutexes or layers of indirection that don't look necessary to me.

If you allow requirements like these to propagate unchecked through a codebase, they will petrify the system over time. The whole thing will be such a mess of checking and re-checking and holding locks and enforcing consistency that it cannot be scaled at all. Keeping systems elegant requires a ruthless focus: on accepting stale data or occasional tail latency, on avoiding all but critical instances of coordination, and in general on figuring out what requirements you *don't* have to satisfy.

Please for the love of all that is holy: Stop writing long boring Titles

SWYX · 30 MAR 2026 · SOURCE

People code switch. Every ethnic minority has lived experience of this. When you talk to me in Singapore, I sound very different than when I'm in San Francisco. And the way you talk at work is probably very different from how you talk to your kids and how you talk to your lover.

I've been dealing with a particularly virulent form of code switching in the simple form of titling blogposts and talks. When you ask the writer face to face what their thing is about, they can give you a perfectly human explanation. And then when they send you their piece, it's all "***Leveraging GenAI with Robust Data Foundations to Transform Businesses***". I've left my body by the time I read the first word.

My hope in writing this piece is to **put an end to bad titles**.

If I sent you this guide, it is not an insult. You just don't have good title game. That's ok. You're great at what you do, title game isn't your job. I'm the guy that wrote How To Name Things and Two Words. I sent you this guide ***because*** your shitty title is the main thing stopping your readers from getting to the good part. If you didn't have anything good, I wouldn't bother with a lost cause.

First…

Towards Leveraging Corporate Speak For Maximum Enterprise Synergy

DO NOT:

- Start with "Leveraging" or "Towards" or "Navigating" or "Empowering"

Yeah just stop. There's no rescuing this when you start there. Just delete meaningless corporate weasel words. Anything >3 syllables is a red flag.

Make It Quotable

If you do not try to make your title roll off the tongue, it probably won't.

Literally imagine telling your friend about your talk. Telling a new acquaintance to google your blogpost. Having a fan of yours cite it back to you. How much do they stumble? Yeah. This has all happened to me.

There's an extremely high correlation between what are considered the best talks and the quotability of titles. Simple Made Easy. Inventing on Principle. Our list of best demos/pitches of all time.

Resist colons. It lacks confidence. Why Use Lot Word: ~~When Few Word Do Trick~~? You're not High School Musical: The Musical: The Series.

Put the SEO keyword vomit in the description.

Titles are for humans.

Prescribe A Strong Opinion

Opinions are one of the most quotable things.

But more importantly, a good Opinion Piece is load bearing because it is eminently forwardable, and therefore viral and impactful. Your Job To Be Done is to be the definitive, conclusive, piece on that Opinion. Stake out territory in Opinion Space, list out all the arguments for, counter all the arguments against, provide datapoints, and watch that go further than anything you've ever done.

How to know what's a good opinion? **Three Strikes rule**: If you repeat yourself three times, ship it. Even if you're just quoting others, its fine, credit them and make it your own.

Mental Models and Frameworks are more complex opinions that allow others to scaffold their thought around you. My Radiating Circles and Measuring Developer Relations and Third Age of JavaScript did this.

Opinions contrary to an established opinion are the hardest to do well and the best when correct. What Clayton Christensen Got Wrong made Ben Thompson's career. Rise of AI Engineer countered the prevailing Prompt Engineer -and- GPU Rich Startup/ML Engineer narrative at the time, with room to grow.

Curiosity Gap Dynamics

Marketers know the power of the Curiosity Gap. Here is where we toy closest with clickbait. I'll be honest that I wasn't even sure if I wanted to give it its own subheading, but its a very important concept that you should at least be aware exists, even if you're going to ignore it.

Strong enough opinions create their own curiosity gaps; you won't have to try very hard to make it interesting if you got my attention in 4 words anyway.

Curiosity gaps done poorly can lead to "burying the lede", which make your talk less Quotable and you also don't want to do that.

Simplest way to put it is; if the most interesting thing about your content is your title, and what people take away after having read your post matches what they would have predicted after reading your title, then you've probably said too much and could cut.

Play Madlibs

You don't have to invent your title from scratch (nor should you always do what I'm about to suggest). Here's some templates that work well:

- {*BENEFIT EVERYONE WANTS*} **without** {*DOWNSIDE EVERYBODY EXPECTS*}
  - e.g. How to Get Rich (Without Getting Lucky), How to Market Yourself (Without Being A Celebrity). See also subtitle of this post.
- {*PRESCRIPTIVE OPINION*}, **don't** {*THING YOU DO*}
  - e.g. Parse, don't validate
  - alternative prescriptive opinion: **Never** {THING YOU DO}
  - see also title of this post
- {*POPULAR THING*} **IS DEAD**
  - unfortunately this is clickbait that works
  - e.g. Prompt Engineering is Dead, Big Data is Dead
  - when given at a conference for *POPULAR THING*, this is known as a Heresy Talk
  - opposite also works:
    - **The Rise of** {THING}
    - **The Unreasonable Effectiveness of** {THING}
- **Fundamentals of** {THING}
  - straightforward, people love fundamentals
  - also **How** {THING} **Really Works**
  - You can also flip it and go for the **Advanced** {THING} tier content

- {CONVEY EXPERIENCE/SCALE} with {THING}
    - e.g. What We Learned from a Year of Building with LLMs, Insights from over ***10,000*** comments on "Ask HN: Who Is Hiring" using GPT-4o

There's loads more templates that work, please suggest in comments and I can add/review.

You can also browse the masters of microcopy like Steph Smith and Shaan Puri, and if you are worried about clickbait, you can see Veritasium's take or Simon Willison's Highlights.

also Andrew Chen senpai:

Understand Buzzword Metagame

Buzzwords work. Because of a mix of human psychology and algorithmic recsys. Don't avoid them, just use them judiciously. This involves the ability to read the room, which many smart people sadly cannot without significant involvement in the broader community you are trying to speak to. I don't see any way around it other than trying to authentically immerse, or consult a friendly who does.

Not all buzzwords are created equal. Databricks/Berkeley tried to push Compound AI Systems, Gartner is pushing Composite AI and Composable business — this resonates with their audience, but not more broadly.

Pick the shibboleths that work for the community you want. AI Engineer worked for reasons that are well understood now. GraphRAG is working outside of the company that coined it. These buzzwords travel, go ahead and put them in there.

Buzzwords rise and fall. Try to catch them on the rise — go harder than everyone else on the rise up and regularly test their effectiveness, such that by the time your main buzzword has tapped out you've got 2-3 others cooking.

Workshopping your titles

I write this post out of frustration, but also to capture my own thought process. I'm currently procrastinating while writing a Latent Space post.

- The original title was Why every AI Engineer needs an AI Gateway. Good, genuine opinion, load bearing. But not super memorable.

- My next iteration was “**Stop writing model routers, use an LLM Gateway**”. Ok, prescriptive, evocative. But didn’t cover the other elements other than model routing. One could shorten to “Stop writing your own LLM Gateway” but that doesnt quite express what people do.

- My next one is “**It's not your job to DIY an LLM Gateway**”. No buzzword bc “your job” is implied (for a Latent Space audience), a little curiosity gap, LLM buzzword, strong opinion. But still not strong enough.

- Final one is “**LLM Gateway: The One Decision That Removes 100 AI Engineering Decisions**”. Leans on Tim Ferriss’ madlbs.

It’s very common for the “tail to wag the dog to wag the tail” - the title is so important to clickthrough and so impactful to the message, that you spend a bunch of time on just the title itself, because that determines the piece you will write, then write the piece, then you go back and tweak the title based on what happened.

Amateur writers think their content has 95% of the value and then spend 5% on the title. Pros will put the value of titles closer to 50% and then act like it.

You Should Break The Rules Once You Understand Human Behavior More Intuitively Than Rules Can Model

See title of this post. No better aura than intentional, conspicuous, successful rulebreaking.

Subscribe to our newsletter

Ownership

THORSTEN BALL · 08 JUL 2026 · SOURCE

Thoughts on ownership I sent to the Amp team in internal note

The following is an internal Slack message I sent to my Amp teammates after I had a conversation with one of them about ownership. It's only lightly edited.

I shared it before, but not here, because I didn't think too much of it. Then today, someone said their CTO shared my post with them and I thought: well, now I have to put it in the newsletter, don't I? So here we are.

Below, after the Slack post, I added some thoughts on juniors on how I see this advice applying to them.



Just had a (great) conversation about ownership and engineering here and I realized that I often use the phrase "ownership" or allude to it, but haven't explained what "ownership" means to me in a while.

So, ownership.

If I ask you "can you own this?" or "can you take care of this?" or "are you on it?" — what I'm doing is I'm asking you to *own* it, to own the solution of a problem from end to end. From "we have a problem" to "we don't have to think about it again."

That means, when you say that you're owning something, the expectation is that you…

- Think about what the problem actually is. Maybe you already have a solution in mind, without having thought about what we're actually trying to solve here. Maybe you think "the problem is that we need to migrate from using X to using Y", but that's not a problem, that's a solution. The problem is likely something like "performance is bad", "it's not stable", "it fails for customer x". Maybe there's other possible solutions to that? Think about those. What are the tradeoffs? What's the best solution to go with considering these tradeoffs?

- Think about edge cases. What are they? Which ones are important? Which ones can we ignore?

- Think about failures. Network failures are a given, for example. How do we handle them? Retry? Well, how often? How long?

- Think about data flow. How much data is involved here? Does data need to be migrated? Cleaned up? How can I get my hands on data to properly test this? What invariants are in the data? What assumptions do I have about the shape of the data that I haven't confirmed yet?

- Think about how you'd test this. How can I know that what I built is correct or not? Are tests enough? Do I need to manually poke at things? Is the difference visible on a screenshot or in a video?

- How would we announce this? How do we communicate it? Can you picture it? How does it fit into the larger picture of our roadmap? Questions or concerns in that area — push back! ask!

- Do the work, with precision, with care, with a sense of urgency, with calmness. Do not half-ass things. Before you merge, ask yourself: am I proud of this? would I show this to John Carmack and say "here's what I built, under these constraints, with these tradeoffs?"

- Test it manually. Yes, there's automated tests. But in 99% of cases you can manually test or confirm that what you built works: you can run it yourself, you can ask an agent to run through test scenarios, you can poke at the data before and after, you can take screenshots, you can make a demo. Are you sure that what you did actually solves the problem?

- Make sure it lands in production and *works in production*. Is it deployed? Did the deploy fail? Do you need to activate a feature flag? Does the feature flag work? Can you use it in production? Can you confirm it's actually deployed?

- If you think your colleagues needs to know about this change, because it's new feature they should all test, or it's a new convention in codebase, or maybe it's a tricky thing everybody needs to be aware of, or something else: let them know! Do not underestimate peripheral vision: knowing that person X yesterday changed the behavior of how Z works might save person Y three hours of debugging today when a bug report related to Z comes in.

- Do customers need to know? Who reported the bug? Who's blocked? Let them know.

- Does the world need to know? Announce that it's out.

- Are there follow-ups? Do you need to check on what you shipped in the logs? A week later maybe?

Yes, that's a lot. And there's actually more, because I'm sure I forgot some stuff.

But that's how you build a product in a small team. We don't have PMs, we don't have a Q&A

department. We're small, but we're *great*, we can do all of that.

And it's always okay to ask for help, it's okay to ask questions, it's okay to redo things and triple-check. What's not okay is to implicitly assume that someone else will do the things here that you haven't thought about.



"How does this apply juniors? You can't expect them to really do all of that?"

I've been asked these questions, or variations of them, after I shared the thoughts above and here's my answer.

I do *not* expect juniors to *do* all of these things right away. But I would expect them to read through the list and *aspire* to one day be able to do all of these things. Until then, they can and should ask for help.

In fact, I don't expect *anybody* to *always* do *all of these things* for *everything*. It's a mental checklist of things to consider — problem, edge cases, tradeoffs, deployment, customers, messaging, … — but for quite a few things there aren't edge cases to consider. Or big tradeoffs to weigh. Or deployment is a solved problem. And maybe someone else does the messaging for you.

And I imagine that most of these things you shouldn't even consider when you work in a company with, say, 5000 employees. I've never worked in a company that large, only startups, so I can't speak to how to successfully ship a software feature at Apple, end to end.

But when you work in a small company in which there's only a single department, when you want to build things you're proud of, when you work with me and you say own something, I expect you to keep these things in mind and run through them before you declare something as done.



You know what *you* should own? A subscription to this newsletter:

The lies I used to tell myself

CATE HALL · 23 JAN 2026 · SOURCE



1. What doesn’t kill you makes you stronger.

When I was 31, my husband of four years, who I loved more than my own life, left me completely out of the blue. We’d had an incredible, epic romance — drawn together like magnets, we married on our third date and spent the years that followed burrowing ever closer together. Maybe that was the problem — I can’t say for sure, because I never got a satisfying explanation out of him. One day I was the protagonist of a Princess Bride-worthy love story; 48 hours later, he moved out.

From the time we met, I had never contemplated the possibility of living without him, and I didn’t want to. I wanted to die for a long time — at least a couple of years. But as I slowly learned to sequester away thoughts of him and the emotions they brought up, I became convinced of a silver lining: The experience would make me impenetrable. If I survived it, I would know I could survive anything, and nothing would be able to hurt me again.

It took the better part of a decade and another deeply identity-shifting relationship, with the man who became my second husband, for me to see that I had it all wrong: Impenetrability means avoiding the parts of life that are most life-like. There’s a reason the visual metaphor for love is an arrow.

The near-fatal catastrophes of our lives don't always make us stronger. Surviving one hard thing does provide evidence that you can survive other hard things, which is easy to *mistake* for getting stronger. But in reality they often just make us brittle, make us dissociate and flee from the situations that remind us we can be broken.

2. When you know, you know.

Mario Gabriele of the Generalist had a great "50 Things" style post a few months ago that I found myself nodding along to almost the whole way through. The glaring exception was Number 33: "Love is easy. The difference between bad relationships and meeting your spouse is not a matter of degree, but category. Compatibility is unignorable and effortless."

I believed this completely, once. I've since come to believe that what's unignorable and effortless is chemistry. Chemistry can be powerful enough to turn your life sideways, but it's not the same thing as compatibility. Compatibility is litigated on the timescale of years and decades; it's not a momentary feeling but a description of what happens during an extended course of change. Do you grow toward one another, or do you grow apart? Does the relationship help you move closer to the person you aspire to be?

I "just knew" with my first husband. I wasn't sure with my second. Four years in, the signs have flipped.

3. If it were a good idea, someone would be doing it already.

There's a belief, especially common among wonky types, that you don't find $20 bills on the sidewalk — if an opportunity were real, someone would have seized it already. This sounds worldly and wise, but it's actually a form of defensive thinking. It assumes the current way is best, that everything worth doing is being done.

In my experience, this is just false. When I started playing poker, I discovered that physical tells were basically a wide-open field — despite the fact that most pros believed the topic was played out. If you think you see an opportunity others have missed, you should have a story for why they're missing it, and you should ask around to stress-test that story. But if no one can give you a satisfying reason, don't assume one exists. Sometimes the answer really is just inertia or fear.

4. If it's worth doing, it's worth doing well.

Almost nothing is worth doing "well." Many things can and should be done to a minimal standard of quality, because anything more is a waste of effort that can be better allocated. The exceptions aren't things that should be done "well," but things that should be done to an exceptionally high standard, probably a much higher one than you see modeled by the people around you.

This is not semantics; most people really "do everything the way they do anything" — whether that's in a low-effort or high-effort way. But the point of phoning it in on a lot of stuff is to save your energy to expend on the 1 in 1000 things that really matter … and then to actually spend it.

A recent example: I decided that I want to narrate my own audiobook for *You Can Just Do Things*. When I told a friend in the industry this and asked whether they knew any good coaches to work with, they said that hiring a coach wasn't normal or necessary, because my publisher would decide based on a sample whether I was good enough at reading to do it, and if so give me a producer to work with day-of. But it "1 in 1000" matters to me whether the audiobook is so good no one would ever think of returning it on account of the narration, so I'm not interested in the normal-shaped solution that constitutes doing it "well," I'm interested in the one that causes me to come to mind when someone asks this question. In this case, that means hiring multiple coaches and practicing for months.

5. If you like your job, you're doing something wrong.

Back when I was more alienated from my emotions, I thought about work like this: You have a budget of energy to spend, and the goal is to efficiently convert that energy into impact. You identify the activity that generates the highest impact per unit of energy, and then you spam that button for as many waking hours as you can stomach. Is it any wonder I was always burning out?

What I failed to understand is that energy is not fixed. It's dramatically affected by what you choose to do — some activities increase your energy over time, others deplete it. This feels so obvious to me now that I feel kind of stupid including it here, but I know many people (they are endemic to the Bay Area) who still labor under the false belief. Whether you enjoy what you're doing affects your ability to do it year in and year out, and dismissing this as "selfish" doesn't make it untrue. As my husband puts it, the best use of effortful motivation is to engineer your life to require less of it.

6. “All people are insane. They will do anything at any time, and God help anybody who looks for reasons.”

This line, from Kurt Vonnegut, used to neatly encapsulate the way I understood other people — which is to say, not at all. Trying to figure out why people did what they did, or imagine what they’d do next, seemed like a fool’s errand. I was judgmental about it, too: I assumed this was a problem, not with me, but with everyone else.

I didn’t fundamentally move beyond this belief until recent years, when I came into contact with the Enneagram. I imagine there are other personality typing systems that can do something similar, but the Enneagram was magical for me because it clued me into the fact that people can have very different motivational systems from my own that are still internally coherent and produce good things in the world.

People’s behavior was previously incomprehensible to me because I was imputing my own motivational system to them, trying to figure out what would cause me to act the way they did — essentially, concluding that *their* actions made no sense in light of *my* goals. But people have wildly diverse emotional hardware, which determines what’s rational for them. And my particular personality — independent, moralistic, systematic, challenge-seeking, competitive — is unusual, so I was especially poorly placed to measure others by the yardstick of myself.

This shift in perspective was dramatically good for me and for my relationships with other people. It let me see the ecosystem of psychologies as a kind of miracle — wow, it really does take all kinds — rather than a prompt for cosmic horror. And it gave me confidence that, if I invested in understanding people more deeply, they would eventually cease to be total mysteries, prone to bizarre and unreasonable behavior. As far as I can tell, this is the better part of emotional intelligence.

7. Money can’t buy happiness.

The good version of this advice is: You’ll still have to work at being happy, even if you’re rich. Money in your bank account won’t do it for you, and wealth comes with its own pathologies, like endless status-chasing. But the common gloss — that money stops mattering past some modest threshold — is basically false.

You may have heard there’s research showing money doesn’t buy happiness. That research was explosively popular precisely because it was counterintuitive — it betrayed the commonsense intuition that more money means more freedom, more options, more ability to help people you care about. Turns out it was also riddled with methodological problems. More recent work by Matthew Killingsworth (nominative determinism FTW), using large-scale studies that pinged participants about their happiness at random moments, found steady returns to well-being with

rising income, no plateau in sight. The myth persists because it serves a psychological function: It helps people without money feel better about their situation, and helps the wealthy downplay their advantages. But it's not true.

8. Intuition is fake and rationality solves everything.

I used to be horribly judgmental of people who made decisions based on intuition, as in, "I just do what feels right." And then I noticed that whenever I overrode my intuitions, I made *terrible* decisions. I hired the wrong people, started projects with the wrong people, dated the wrong people, ate food that made me feel bad, did forms of exercise I found unfun and injurious. I don't have a satisfying theory for why this is, but I know what happens: When I shift from emotional intuition to a logical story about a decision, it's much easier for my reasoning to get hijacked by plausible-sounding directives that turn out to be nonsense.

It took me a long time to learn this, and that might have something to do with my training in law and poker. Overriding intuition in favor of explicit reasoning makes sense in contexts that reward systematic rationality, where you will get separated from your money if you count on gut feelings. But most of life isn't a closed system with enumerated rules and clear outcomes. If you try to explicitly name all the factors that make someone a good friend or co-founder, you might come up with some good indicators, but your crude map will also mislead you about the territory.

This is why "wisdom is knowledge that can't be transmitted." Wise people have navigational skill, not a long list of rules.

Sign up to be notified when my book, You Can Just Do Things, is available for purchase.

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · SOURCE

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments…). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs… but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding doc saying "NEVER TOUCH STATE IN FOOBARDB DIRECTLY!". You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool's existence, or maybe it just preferred FooBarDB's in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name "delete-everything". Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases

(Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!